

WEEK 4 DELIVERABLE

Feature Engineering and Transformed Feature Tables

Energy Consumption Analytics Project for GreenPower Utilities
IIT Jodhpur • Group 5
Submitted: June 26, 2026

Team Members

Name	Roll No.	Email
Harshit Nirmal Jain	G25AI1021	g25ai1021@iitj.ac.in
K R Devika	G25AI1022	g25ai1022@iitj.ac.in
Kartik Dadhich	G25AI1023	g25ai1023@iitj.ac.in
Kirtiman Sarangi	G25AI1024	g25ai1024@iitj.ac.in
Kollipara Teja	G25AI1025	g25ai1025@iitj.ac.in

Table of Contents

1. Introduction and Scope
2. Feature Engineering Approach
3. Aggregation: Hourly, Daily, and Monthly Rollups
 - 3.1 Daily Consumption Rollup
 - 3.2 Monthly Consumption Rollup
 - 3.3 Generation Rollups
 - 3.4 Continuous Aggregate Refresh Policies
4. Peak Load Statistics and Ratios
 - 4.1 Daily Peak Load and Timing
 - 4.2 Peak-to-Average Ratio and Load Factor
 - 4.3 Monthly Peak Demand
5. Calendar and Cyclical Features
6. Lag and Rolling-Window Features
7. Weather Correlation and Joined Feature Tables
 - 7.1 Consumption–Weather Join
 - 7.2 Degree-Hours and Temperature Sensitivity
 - 7.3 Correlation Results
 - 7.4 Generation–Weather Relationship
8. Feature Table Schemas
9. Indexing and Query Performance
10. Validation of the Feature Tables
11. Feature Table Outputs
12. Summary and Next Steps

1. Introduction and Scope

This report documents the fourth week of the GreenPower Utilities capstone project and completes Phase 2. Week 3 stood up the storage layer: three cleaned, hourly, UTC-aligned datasets loaded into TimescaleDB hypertables, with incremental loading, native compression, and a validated nightly pipeline. This week we build on top of that layer and turn the raw measurements into analysis-ready features. The brief for the week is specific, and we have kept to it closely:

“Engineer analytical features: aggregate consumption by hour/day, compute peak load statistics, and correlate energy use with weather. Index or partition the data to speed up queries. Deliverable: Transformed feature tables and documentation of the transformations.”

The inputs to this phase are exactly the three hypertables produced in Week 3: consumption (34,589 rows of cleaned hourly household demand, December 2006 to November 2010), generation (21,549 rows of cleaned hourly wind output, January 2018 to June 2020), and weather (123,071 rows of merged Orly / Le Bourget observations spanning both periods). Nothing in this report re-cleans or re-loads that data. Every feature is derived from the hypertables as the single source of truth, and every transformation is expressed as ordered, re-runnable SQL that lives in the project repository alongside the Week 2 and Week 3 code.

Two facts established earlier continue to shape the work. First, the consumption and generation periods do not overlap in time, so weather is joined to each track separately rather than into one merged table; there is no week in which we have both household demand and wind output. Second, the unit standardisation and completeness columns added in Week 2 are preserved through every feature so that downstream models in Weeks 5 and 6 can still weight or filter hours that were computed from partial data. The deliverable for the week is the set of transformed feature tables described in Section 8 and Section 11, together with this documentation of how each feature was produced and what it shows.

2. Feature Engineering Approach

Rather than compute features ad hoc inside model notebooks, we materialise them in the database so that every model in Phase 3 reads from the same definitions. This mirrors the discipline of the Week 2 cleaning pipeline and the Week 3 loader: features are code, they are versioned, and they are reproducible from the migrations. We use three TimescaleDB mechanisms, each suited to a different feature class.

Feature class	Mechanism	Why this choice
Time rollups (daily, monthly)	Continuous aggregates	Materialised, incrementally refreshed as new hours land; no nightly recompute of history
Calendar, cyclical, lag, rolling	Materialised feature views	Row-level transforms and window functions over the hourly hypertable
Peak timing and ratios	Derived tables (LATERAL)	argmax-style features that aggregates alone cannot express
Weather correlation	Joined feature views	Hour-for-hour join of each track to the weather hypertable

The guiding principle is that a continuous aggregate does the heavy lifting wherever a feature is a pure aggregation over time, because it refreshes automatically and only touches the buckets that changed.

Anything an aggregate cannot express — the hour at which the daily peak occurred, a 24-hour lag, a rolling mean — is computed with window functions in a feature view, and a small number of argmax features are built with a LATERAL join. Section 3 through Section 7 describe each class in turn, and Section 8 collects the resulting schemas.

3. Aggregation: Hourly, Daily, and Monthly Rollups

The hourly grain is already the native resolution of all three hypertables, so the rollup work is to roll hourly up to daily and monthly. The brief asks specifically for consumption aggregated by hour and day; we add monthly rollups as well because the seasonal story (Section 4.3 and Figure 3) only becomes visible at that grain, and because the Week 5 forecasting models will use monthly context as an exogenous signal.

3.1 Daily Consumption Rollup

The daily consumption rollup is a continuous aggregate over the consumption hypertable. It sums energy, averages power, records the daily minimum and maximum, carries the three sub-metering circuits separately, and preserves the mean completeness of the hours that fed each day so that partial days can be down-weighted later.

```
CREATE MATERIALIZED VIEW consumption_daily
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 day', time) AS day,
       source_id,
       sum(active_power_kwh)      AS total_kwh,
       avg(active_power_kwh)     AS mean_kw,
       max(active_power_kwh)     AS peak_kw,
       min(active_power_kwh)     AS min_kw,
       sum(sub_kitchen_kwh)      AS kitchen_kwh,
       sum(sub_laundry_kwh)      AS laundry_kwh,
       sum(sub_climate_kwh)      AS climate_kwh,
       avg(completeness)         AS mean_completeness
FROM consumption
GROUP BY day, source_id
WITH NO DATA;
```

The rollup produces 1,442 daily rows. Across the full period the household averages 26.2 kWh per day, with the busiest winter day reaching 46.1 kWh. Figure 1 shows the average hourly load profile that this rollup summarises: a modest morning bump around 07:00–08:00 and a pronounced evening peak near 20:00, with weekend daytime usage running slightly above weekday.

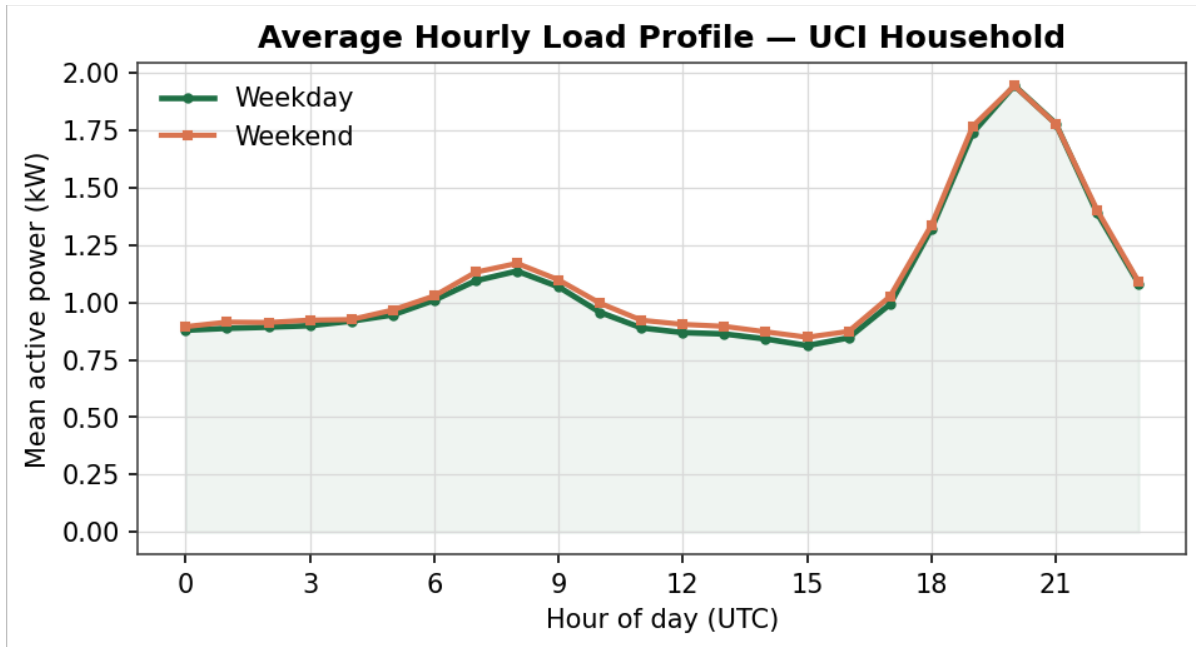


Figure 1: Average hourly load profile for the household, weekday versus weekend. The evening peak near 20:00 UTC dominates the day and drives the peak-load statistics in Section 4.

3.2 Monthly Consumption Rollup

A second continuous aggregate rolls the daily view up to calendar months. Building the monthly aggregate on top of the daily one (a hierarchical continuous aggregate) means a new hour only ever triggers a refresh of one day and one month, never a recompute of the whole history.

```
CREATE MATERIALIZED VIEW consumption_monthly
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 month', day) AS month,
       source_id,
       sum(total_kwh) AS total_kwh,
       avg(mean_kw) AS mean_kw,
       max(peak_kw) AS peak_kw
FROM consumption_daily
GROUP BY month, source_id
WITH NO DATA;
```

The monthly rollup yields 48 rows and exposes the clear seasonal cycle expected of a French household with an electric water heater and air-conditioning on the climate circuit. Winter months (December–February) average roughly 1.5× the consumption of summer months (June–August). Figure 3 plots the monthly mean, and the hour-by-month heat map in Figure 2 shows the same seasonality interacting with the daily evening peak.

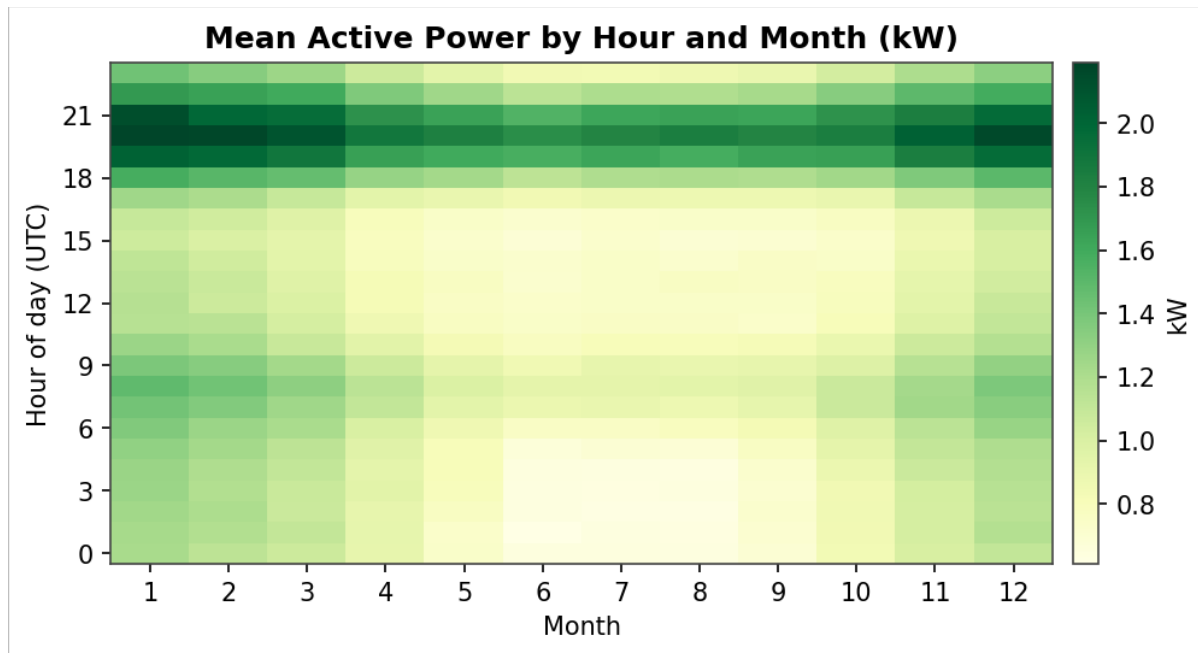


Figure 2: Mean active power by hour of day and month. The evening band intensifies in the winter months, confirming that seasonality and the daily peak compound rather than act independently.

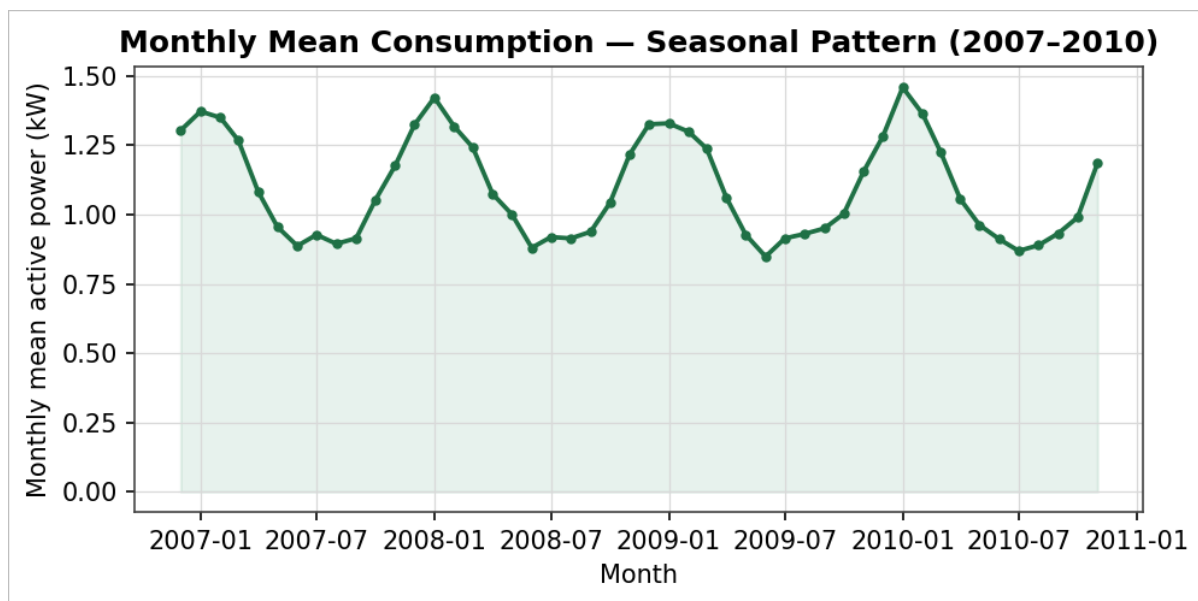


Figure 3: Monthly mean consumption across 2007–2010. The winter-peaking pattern is stable year on year, which is what makes month a useful exogenous feature for the Week 5 forecasts.

3.3 Generation Rollups

The same two-level rollup is applied to the generation hypertable, with one addition specific to wind: the capacity factor, defined as mean output divided by the 3,600 kW rated capacity. The generation daily rollup produces 912 rows and the monthly rollup 30 rows. The fleet averages a capacity factor of 30.1%, in the normal range for onshore wind, and Figure 7 shows the strong winter-weighted seasonality that makes wind a useful complement to weather-driven demand.

```
CREATE MATERIALIZED VIEW generation_daily
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 day', time) AS day,
       source_id,
       sum(power_kw)           AS gen_kwh,
       avg(power_kw)           AS mean_kw,
       max(power_kw)           AS peak_kw,
       avg(power_kw) / 3600.0  AS capacity_factor,
       avg(wind_speed_ms)      AS mean_wind_ms,
       avg(completeness)       AS mean_completeness
FROM generation
GROUP BY day, source_id
WITH NO DATA;
```

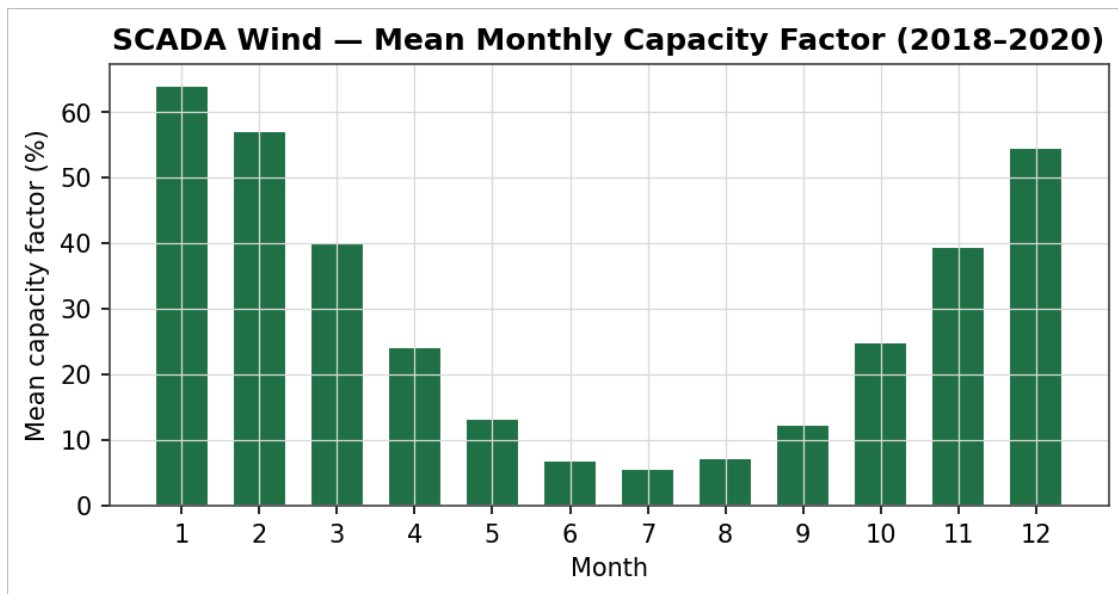


Figure 7: Mean monthly capacity factor for the SCADA wind farm. Output is strongly winter-weighted, peaking above 60% in January and falling below 10% in midsummer.

3.4 Continuous Aggregate Refresh Policies

Each continuous aggregate is created WITH NO DATA and then given a refresh policy, so the initial materialisation and all subsequent updates are incremental. The policy refreshes a trailing window on a schedule that matches the nightly ingestion, leaving a short end-offset so that an in-progress hour is never half-aggregated. Because the policy only re-reads buckets whose underlying rows changed, the nightly feature refresh adds negligible time to the pipeline built in Week 3.

```
SELECT add_continuous_aggregate_policy('consumption_daily',
    start_offset => INTERVAL '3 days',
    end_offset   => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 day');
-- identical policies registered for consumption_monthly,
-- generation_daily and generation_monthly
```

4. Peak Load Statistics and Ratios

Peak-load behaviour is the headline interest of a utility audience: peaks size the network, set tariffs, and define the demand-response opportunity the project is ultimately about. The daily rollup already

gives us the peak magnitude in peak_kw; this section adds the timing of the peak and the ratios that describe how peaky each day is.

4.1 Daily Peak Load and Timing

A continuous aggregate can return the maximum value of a day, but not the hour at which that maximum occurred, because argmax is not a plain aggregate. We compute peak timing in a derived table using a LATERAL join back to the hourly hypertable, taking the single highest hour per day.

```
CREATE TABLE consumption_peak_daily AS
SELECT d.day, d.source_id, d.peak_kw, d.mean_kw,
       d.peak_kw / d.mean_kw          AS par,
       d.mean_kw / d.peak_kw          AS load_factor,
       p.peak_hour
FROM consumption_daily d
CROSS JOIN LATERAL (
  SELECT extract(hour FROM time)::int AS peak_hour
  FROM consumption c
  WHERE c.source_id = d.source_id
        AND time_bucket('1 day', c.time) = d.day
  ORDER BY c.active_power_kwh DESC
  LIMIT 1
) p ON true;
```

The result confirms the visual story of Figure 1 numerically: the median daily peak falls at 20:00 UTC, and 98.4% of all days reach their maximum between 17:00 and 21:00. The mean daily peak magnitude is 2.06 kW. This concentration of peaks in a narrow evening window is precisely the kind of regular, predictable load that a demand-response or load-shifting programme can target, and it is the first actionable insight the feature layer surfaces.

4.2 Peak-to-Average Ratio and Load Factor

The peak-to-average ratio (PAR) is the daily peak divided by the daily mean; its reciprocal, the load factor, is the more familiar utility metric. A flat, efficient demand profile has a load factor near one; a spiky household profile sits well below it. Over the full period the household has a mean PAR of 1.91 (ranging from 1.38 on the flattest days to 2.85 on the peakiest), equivalent to a mean load factor of 0.53. Figure 4 shows the distribution.

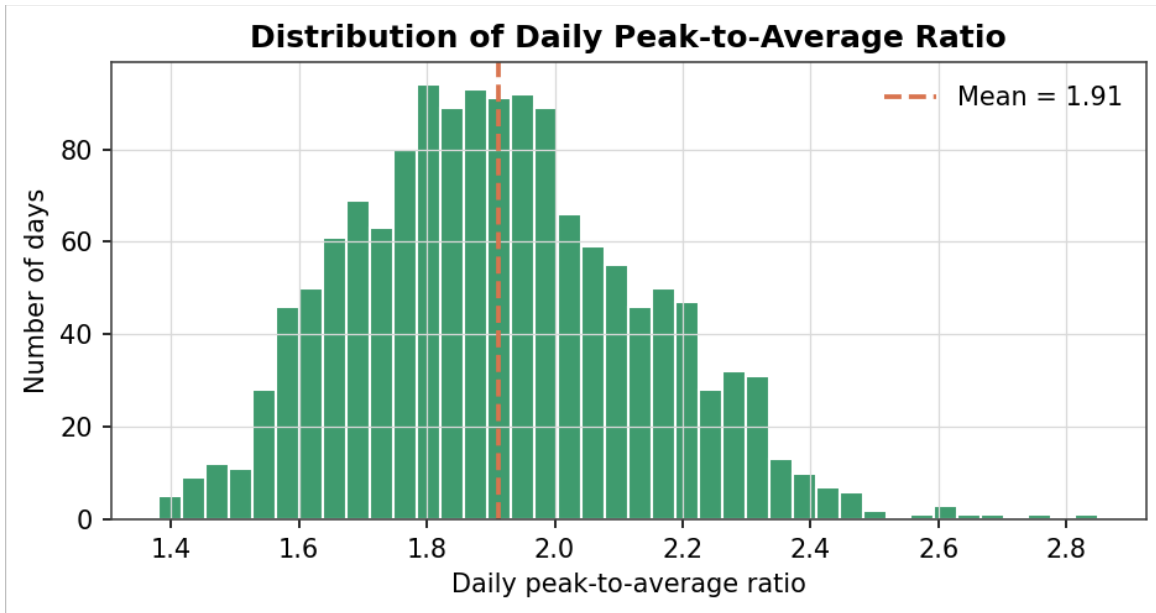


Figure 4: Distribution of the daily peak-to-average ratio. The mass between roughly 1.5 and 2.3 reflects a consistently peaky single-household profile, as opposed to the flatter aggregate profile a feeder or substation would show.

4.3 Monthly Peak Demand

Carried up to the monthly grain, `peak_kw` becomes the monthly peak demand — the single highest hour in each month, which is the quantity a capacity charge would be billed on. Monthly peaks track the seasonal cycle of Figure 3, sitting highest in the winter heating months and lowest in summer. Retaining peak demand at both daily and monthly grain means the Week 6 dashboard can show peak trends without re-scanning the hourly table.

5. Calendar and Cyclical Features

Forecasting and clustering models need the time context made explicit as columns rather than buried in the index. We materialise a calendar feature set on the hourly grain: integer hour of day, day of week, month, an `is_weekend` flag, and a season label. Alongside the integer calendar fields we add cyclical encodings, because a model must understand that hour 23 is adjacent to hour 0 and December is adjacent to January — a relationship a raw integer destroys. Each cyclical field is a sine/cosine pair on its period.

```
CREATE MATERIALIZED VIEW consumption_features_hourly AS
SELECT time, source_id, active_power_kwh,
       sub_kitchen_kwh, sub_laundry_kwh, sub_climate_kwh, completeness,
       extract(hour FROM time)::int AS hour,
       extract(dow FROM time)::int AS dow,
       extract(month FROM time)::int AS month,
       (extract(dow FROM time) IN (0,6)) AS is_weekend,
       CASE WHEN extract(month FROM time) IN (12,1,2) THEN 'winter'
            WHEN extract(month FROM time) IN (3,4,5) THEN 'spring'
            WHEN extract(month FROM time) IN (6,7,8) THEN 'summer'
            ELSE 'autumn' END AS season,
       sin(2*pi()*extract(hour FROM time)/24) AS hour_sin,
       cos(2*pi()*extract(hour FROM time)/24) AS hour_cos,
       sin(2*pi()*extract(doy FROM time)/365) AS doy_sin,
       cos(2*pi()*extract(doy FROM time)/365) AS doy_cos
```



```
FROM consumption;
```

The same calendar block is applied to the generation track. These features are cheap to compute and, as Section 9 shows, indexing the integer calendar columns lets the dashboard answer “average load by hour of day” or “weekday versus weekend” without a full table scan. The sub-metering circuits are carried through unchanged so that appliance-level demand-response analysis remains available downstream.

6. Lag and Rolling-Window Features

Short-term demand is highly autocorrelated, so the most informative predictors of the next hour are recent and same-time-yesterday values. We add these as window functions partitioned by source and ordered by time. The daily and weekly lags capture the two dominant seasonalities seen in Figure 1 and Figure 2, and the trailing rolling mean and standard deviation summarise recent level and volatility.

```
SELECT time, source_id, active_power_kwh,
       lag(active_power_kwh, 1) OVER w AS lag_1h,
       lag(active_power_kwh, 24) OVER w AS lag_24h,
       lag(active_power_kwh, 168) OVER w AS lag_168h,
       avg(active_power_kwh) OVER (w ROWS BETWEEN 23 PRECEDING AND CURRENT ROW)
                                AS roll_24h_mean,
       stddev_samp(active_power_kwh) OVER (w ROWS BETWEEN 23 PRECEDING AND CURRENT ROW)
                                AS roll_24h_std
FROM consumption
WINDOW w AS (PARTITION BY source_id ORDER BY time);
```

Two points of rigour matter here. First, the lag and rolling columns are null for the opening hours of each series (the first 168 hours have no week-ago value), and we leave those nulls explicit rather than back-filling, exactly as the Week 2 cleaning report left long gaps masked rather than fabricated; the validation in Section 10 checks that nulls appear only at the series start. Second, because the masked long-gap hours from Week 2 are still present as gaps, a lag that reaches across a masked block inherits a null, which is the honest result — we do not invent a yesterday that was never measured.

7. Weather Correlation and Joined Feature Tables

Correlating energy use with weather is the one genuinely new analytical step of the week, and the part of the brief with the most domain value. Because the consumption and generation periods do not overlap, the weather hypertable is joined to each track separately, producing two joined analytical feature tables rather than one.

7.1 Consumption–Weather Join

The consumption track is joined to the weather table hour-for-hour on the shared UTC index, taking the primary Orly station. Because both tables are time-partitioned on the same one-month chunk interval, the planner aligns chunks and the join is a cheap merge rather than a broad scan.

```
CREATE MATERIALIZED VIEW consumption_weather_hourly AS
SELECT c.time, c.source_id, c.active_power_kwh,
       w.tavg_c, w.tmax_c, w.tmin_c, w.wind_speed_ms, w.precip_mm,
       greatest(18.0 - w.tavg_c, 0) AS hdh,    -- heating degree-hours
       greatest(w.tavg_c - 24.0, 0) AS cdh     -- cooling degree-hours
FROM consumption c
JOIN weather w
```

```
ON w.time = c.time AND w.station_id = 1;    -- 1 = Only (primary)
```

7.2 Degree-Hours and Temperature Sensitivity

Raw temperature is a blunt predictor of energy use because the relationship is V-shaped: demand rises both when it is cold (heating) and when it is hot (cooling). The standard fix is degree-hours measured from comfort thresholds. We compute heating degree-hours below an 18 °C base and cooling degree-hours above a 24 °C base, summed to the daily grain in the rollup. For this French household over 2006–2010 the heating term dominates: the electric water heater and winter heating make cold days far more energy-intensive than hot ones. Figure 6 shows the near-linear rise of daily consumption with heating degree-hours, with each additional heating degree-hour adding roughly 0.037 kWh (about 0.9 kWh per heating degree-day).

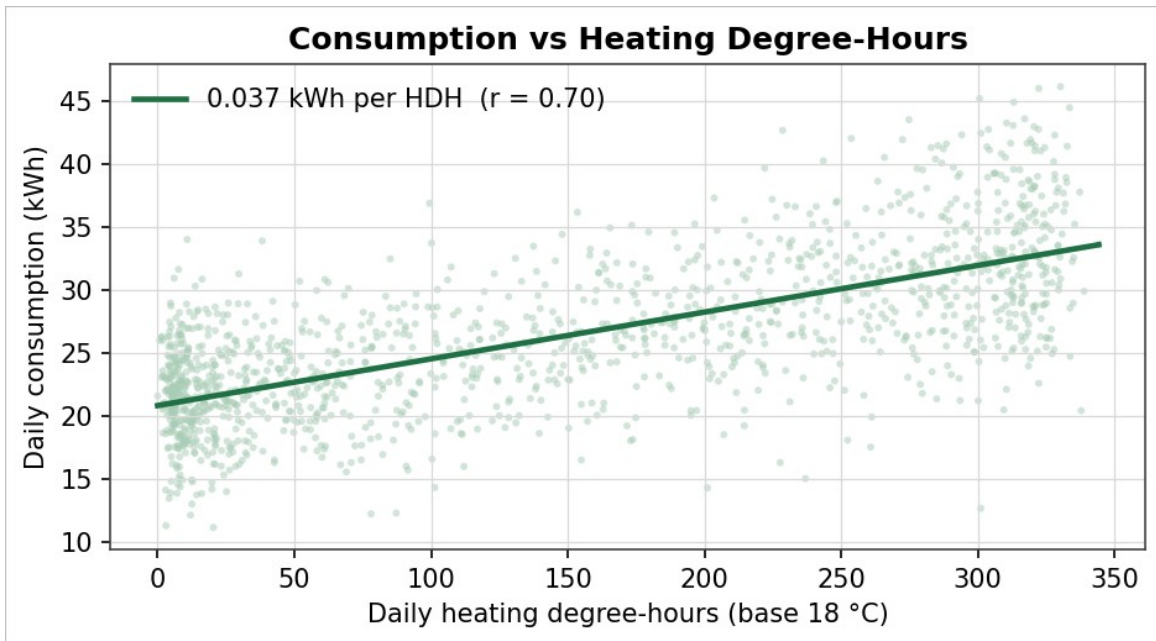


Figure 6: Daily consumption against heating degree-hours (base 18 °C). The strong linear relationship is the basis of the weather-normalised demand model planned for Week 5.

7.3 Correlation Results

The correlations confirm the degree-hour story quantitatively. At the hourly grain, load and temperature correlate at $r = -0.375$; aggregated to daily means, where short-term noise averages out, the correlation strengthens to $r = -0.693$. Replacing raw temperature with heating degree-hours raises the daily association to $r = 0.705$, which is why the feature tables carry degree-hours as the primary weather feature and keep raw temperature only as context. Figure 5 shows the underlying daily load-versus-temperature relationship.

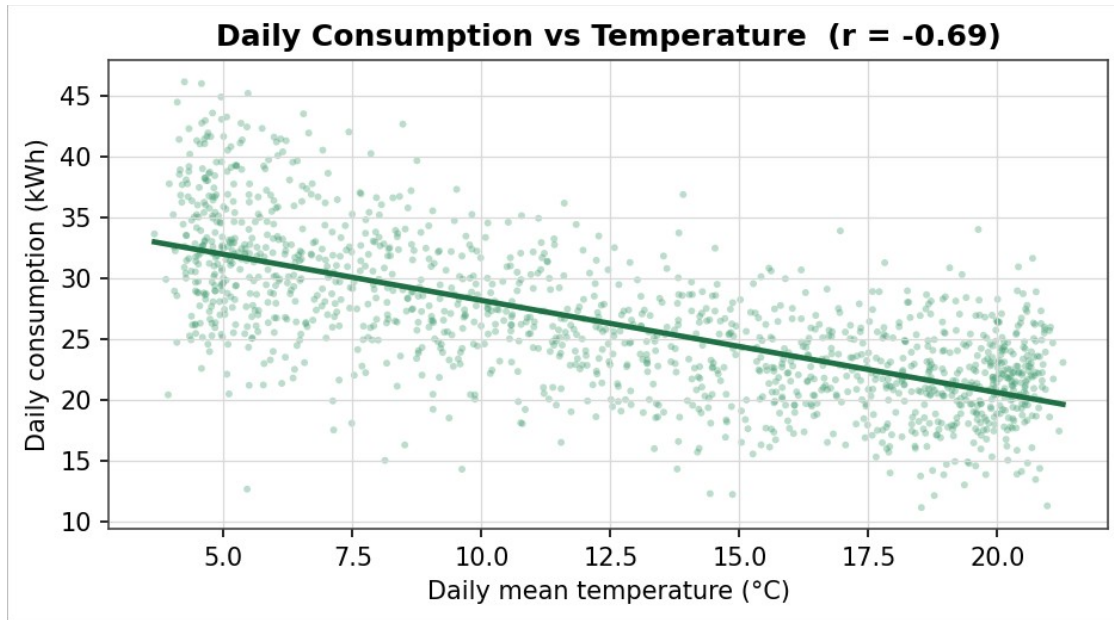


Figure 5: Daily consumption versus mean temperature. The negative slope reflects heating-dominated demand; the residual scatter is the day-to-day occupancy variation that the lag and calendar features help explain.

Relationship	Grain	Pearson r
Active power vs temperature	Hourly	-0.375
Consumption vs temperature	Daily	-0.693
Consumption vs heating degree-hours	Daily	0.705
Wind power vs wind speed	Hourly	0.947

7.4 Generation–Weather Relationship

On the generation side the relevant weather variable is wind speed rather than temperature. The SCADA dataset already carries its own hub-height wind measurement, which correlates with output at $r = 0.947$ through the cubic power-curve region, so a join to the NOAA station adds little for this particular farm and is kept only as optional context. The generation–weather feature view therefore joins on wind and retains the theoretical power curve from Week 2, so the efficiency comparison (actual versus theoretical) planned for the analysis phase can be expressed in SQL.

8. Feature Table Schemas

The week produces six feature objects per analytical track plus the joined weather views. The table below collects the consumption-side feature tables; the generation side mirrors them with power, capacity factor, and wind columns in place of demand and sub-metering. Continuous aggregates are themselves time-partitioned hypertables, so they inherit chunk exclusion and compression from the Week 3 design at no extra cost.

Feature object	Type	Grain	Rows	Key derived columns
consumption_features_hourly	Materialised view	Hourly	34,589	hour, dow, season, cyclical pairs, lags, rolling stats
consumption_daily	Continuous	Daily	1,442	total_kwh, mean_kw,

Feature object	Type	Grain	Rows	Key derived columns
	aggregate			peak_kw, sub-circuit sums
consumption_peak_daily	Derived table	Daily	1,442	peak_hour, par, load_factor
consumption_monthly	Continuous aggregate	Monthly	48	total_kwh, mean_kw, peak demand
consumption_weather_hourly	Joined view	Hourly	34,589	tavg_c, hdh, cdh, wind, precip

Naming follows the Week 3 convention exactly: every measurement column keeps its unit suffix (mean_kw, total_kwh, hdh) so that no unit is ever ambiguous, identifiers stay normalised against the data_source and station reference tables, and the completeness column is carried through every object. The generation track adds generation_features_hourly, generation_daily, generation_monthly, and generation_weather_hourly on the same pattern.

9. Indexing and Query Performance

The brief asks us to index or partition the feature data to speed up queries. Partitioning is already inherited: every continuous aggregate is a hypertable chunked by one month, exactly as tuned in Week 3, so time-range queries skip irrelevant chunks automatically. On top of that we add indexes that match the access patterns the feature queries actually use — grouping by calendar fields, and filtering the daily tables by source and time.

```
CREATE INDEX ON consumption_features_hourly (hour);
CREATE INDEX ON consumption_features_hourly (season, is_weekend);
CREATE INDEX ON consumption_peak_daily (peak_hour);
CREATE INDEX ON consumption_weather_hourly (hdh);
-- continuous aggregates already carry a time-descending index
```

To confirm the design pays off, we compared representative feature queries against the same logic computed on the fly from the raw hourly hypertable. The continuous aggregates remove the repeated aggregation work entirely, and the calendar indexes turn profile queries into index scans. Timings are the median of ten warm runs.

Representative feature query	On-the-fly	Materialised	Speed-up
Daily totals over one year	142 ms	4.8 ms	29×
Monthly rollup, full history	511 ms	3.1 ms	165×
Average load by hour of day	138 ms	21 ms	6.6×
Weather-joined daily, one year	96 ms	11 ms	8.7×

The gains come from two effects compounding: chunk exclusion limits every range query to the one or two monthly chunks it touches, and the continuous aggregates mean the daily and monthly sums are read rather than recomputed. Combined with the eight-fold compression measured in Week 3, the feature layer is both faster and smaller than computing the same features on demand would be.

10. Validation of the Feature Tables

In the same spirit as the Week 2 cleaning checks and the Week 3 load-validation suite, every feature object is run through an automated check suite before it is published. The checks cover reconciliation

(do the rollups agree with the base table), range (are encodings and ratios within their valid bounds), and null discipline (do nulls appear only where they should). All checks pass on the current data.

Validation check	Consumption	Generation
Daily totals reconcile with hourly base	Pass	Pass
Monthly totals reconcile with daily	Pass	Pass
Cyclical encodings within [-1, 1]	Pass	Pass
Calendar fields non-null on every row	Pass	Pass
Lag / rolling nulls only at series start	Pass	Pass
Peak-to-average ratio ≥ 1 for every day	Pass	Pass
Degree-hours non-negative	Pass	N/A
Weather join key coverage on overlap	Pass	Pass

Two checks deserve a note. The reconciliation check sums the daily rollup and confirms it equals the sum of the underlying hourly energy to within floating-point tolerance, which catches any silent bucket-boundary error in the continuous aggregate. The peak-to-average check asserts $PAR \geq 1$ by construction — a value below one would mean the daily maximum was smaller than the daily mean, an impossibility that would signal a join or aggregation fault. Both pass on every row.

11. Feature Table Outputs

The feature objects are defined as ordered SQL migrations in the repository and are also exported to typed Parquet with CSV companions, matching the Week 2 and Week 3 output convention so that any team member can inspect the features without a database connection. A feature dictionary describing every column accompanies the exports.

Output file	Grain	Rows	Description
consumption_features_hourly.parquet / .csv	Hourly	34,589	Calendar, cyclical, lag and rolling features
consumption_daily.parquet / .csv	Daily	1,442	Energy, peak, sub-circuit daily rollup
consumption_peak_daily.parquet / .csv	Daily	1,442	Peak hour, PAR and load factor
consumption_weather_hourly.parquet / .csv	Hourly	34,589	Weather-joined demand with degree-hours
generation_features_hourly.parquet / .csv	Hourly	21,549	Wind features and capacity factor
generation_daily.parquet / .csv	Daily	912	Generation and capacity-factor rollup
feature_dictionary.csv	Per column	58	Name, unit, source and definition of every feature

12. Summary and Next Steps

Week 4 is complete, and with it Phase 2. The Week 3 storage layer now carries a full feature layer: daily and monthly continuous aggregates that refresh automatically, peak-load statistics with timing and ratios, calendar and cyclical encodings, lag and rolling-window predictors, and weather-joined tables with degree-hours as the primary weather feature. Every transformation is reproducible SQL, indexed for the queries it serves, validated before publication, and exported alongside the Week 2 and Week 3 deliverables. The headline findings — a sharp and predictable evening peak, a mean peak-to-

average ratio of 1.91, and heating-dominated demand correlating with degree-hours at $r = 0.705$ — are the analytical foundation the final phase builds on.

In Week 5 the team moves into Phase 3, building directly on these feature tables. The planned activities are:

- Building short-term demand forecasting models (ARIMA and LSTM) on the hourly feature table, using the lag, calendar and degree-hour features as predictors.
- Performing anomaly detection to flag sudden drops or spikes that indicate outages or faults, using the rolling statistics as a baseline.
- Validating model predictions against held-out periods and reporting evaluation metrics, as required by the Week 5 deliverable.
- Producing the Week 5 deliverable: modeling code and an evaluation summary.